

华东师范大学软件工程学院实验报告

姓名：高珩博

学号：10255101445

实验编号：Lab1

实验名称：DataLab

一、实验目的

- 1、位操作- 掌握位运算符的使用，理解整数的二进制位级表示，学习通过位操作实现逻辑和算术功能。
- 2、二进制补码运算- 深入理解 32 位补码表示方法，掌握补码的算术运算规则，理解符号位、溢出等概念。
- 3、浮点数运算- 理解 IEEE 754 单精度浮点数的位级表示格式，掌握符号位、阶码、尾数的作用，学习浮点数与整数的转换及运算规则。

二、实验内容与实验步骤

1、bitAnd

实现：利用德摩根定律，通过两次取反和或运算实现与操作。

```
int bitAnd(int x, int y) {  
    /*  
     * 布尔代数运算  
     * (x & y) 等价于 ~(~(x & y)) 等价于 ~(~x | ~y)  
     */  
    return ~(~x | ~y);  
}
```

2、getBytes

实现：通过右移将目标字节移到最低位，再与 0xff 进行与运算。

```
int getByte(int x, int n) {  
    /*  
     * 使用右移运算符，将目标字节右移到最低位  
     * 再与 0xff 做与运算  
     */  
    return (x >> (n << 3)) & 0xff;  
}
```

3、logicalShift

实现：算术右移会在高位补符号位，需要将高位清零，即先算术右移，然后用和 0 进行与运算清除高位补充的符号位。

```
int logicalShift(int x, int n) {  
    /*  
     * 先右移，由于逻辑右移在左侧补 0，  
     * 故将高位补充的部分都与 0 做与运算  
     */  
    int xx = x >> n;  
    int y = xx & (((1 << (31 + ~n + 1)) << 1) + ~1 + 1);  
    return y;  
}
```

4、bitCount

实现：逐级合并相邻位的计数。

```
int bitCount(int x) {
    /*
     * 先计算每两位中 1 的个数
     * 再计算每四位中 1 的个数
     * 以此类推直到全部位中 1 的个数
     */
    int xx = x;
    int mask_1 = 0x55;
    int mask_2 = 0x33;
    int mask_4 = 0x0f;
    int mask_8 = 0xff;
    int mask_16 = 0xff;

    mask_1 = (mask_1 << 8) + mask_1;
    mask_1 = (mask_1 << 16) + mask_1;
    mask_2 = (mask_2 << 8) + mask_2;
    mask_2 = (mask_2 << 16) + mask_2;
    mask_4 = (mask_4 << 8) + mask_4;
    mask_4 = (mask_4 << 16) + mask_4;
    mask_8 = (mask_8 << 16) + mask_8;
    mask_16 = (mask_16 << 8) + mask_16;
    xx = (xx & mask_1) + ((xx >> 1) & mask_1);
    xx = (xx & mask_2) + ((xx >> 2) & mask_2);
    xx = (xx & mask_4) + ((xx >> 4) & mask_4);
    xx = (xx & mask_8) + ((xx >> 8) & mask_8);
    xx = (xx & mask_16) + ((xx >> 16) & mask_16);

    return xx;
}
```

5、bang

实现：0 和 -0 的最高位都为 0，非 0 数 x 和 -x 中总有一个最高位为 1，故只需判断 x 和 -x 的最高位是否都为零。

```
int bang(int x) {
    /*
     * 只需判断 x 与 -x 的最高位中是否有 1
     */
    return (((x | (~x + 1)) >> 31) & 1) ^ 1;
}
```

6、twin

实现：通过移位构造 0x80000000。

```
int tmin(void) {
    /*
     * 以 2 的补码表示的 32 位整型的最小值 0x80000000
     */
    return 0x80 << 24;
}
```

7、fitsBits

实现：检查 x 的高位是否全部等于符号位。

```

int fitsBits(int x, int n) {
    /*
     * 检查 x 的高位 (x >> (n-1)) 是否等于符号位 (x >> 31)
     * 如果相等, 则 x 可以用 n 位二进制补码表示
     * 对于 n=32, 特殊返回 0
     */
    int shift = n + ~0; // n-1
    int sign = x >> 31;
    int high = x >> shift;
    return !(high ^ sign) & !(n ^ 32);
}
/*

```

8、divpwr2

实现：负数除法需要考虑偏置值，非负数偏置值为 0。

```

int divpwr2(int x, int n) {
    /*
     * 已知补码除法的公式为  $(x < 0 ? x + (1 \ll n) - 1 : x) \gg n$ 
     * 利用符号位和偏置值的与运算，使得非负数的偏置值为 0
     */
    int mask = (x >> 31);
    int bias = ((1 << n) + ~1 + 1) & mask;
    int res = (bias + x) >> n;

    return res;
}

```

9、negate

实现：补码取反等于按位取反后加 1。

```

int negate(int x) {
    /*
     * 取非相当于取反后 + 1
     */
    return ~x+1;
}

```

10、isPositive

实现：正数的符号位为 0 且本身不为 0。

```

int isPositive(int x) {
    /*
     * 一个数为正数当且仅当它的符号位不为 1 且它本身不为 0
     * 通过右移 31 位获取该数的符号位
     */
    int sign = (x >> 31) & 1;

    return !sign & !!x;
}

```

11、isLessOrEqual

实现：通过 $y-x$ 的符号判断，但需考虑溢出情况。

```

int isLessOrEqual(int x, int y) {
    /*
     * 当两数之差超过 TMAX 时会发生溢出 需分两数同号、两数异号两种情况讨论
     * 当两数同号时，不会溢出，直接判断差值的符号位
     * 当两数异号时，可能溢出，直接判断是否是 x 为负且 y 为正
     */
    int dif = (y + ~x + 1);
    int sign_of_dif = (dif >> 31) & 1;
    int signx = (x >> 31) & 1;
    int signy = (y >> 31) & 1;
    int dif_sign = (signx ^ signy);

    return (dif_sign & signx) | (!dif_sign & !sign_of_dif);
}

```

12、ilog2

实现：用二分法逐级缩小范围，找到最高位 1 的位置。

```

int ilog2(int x) {
    /*
     * 求这个数中最高位的 1 的位置
     * 使用二分折半寻找最高位的 1
     */
    int res = 0;
    res = res + (!! (x >> 16) << 4);
    x = x >> (!! (x >> 16) << 4);
    res = res + (!! (x >> 8) << 3);
    x = x >> (!! (x >> 8) << 3);
    res = res + (!! (x >> 4) << 2);
    x = x >> (!! (x >> 4) << 2);
    res = res + (!! (x >> 2) << 1);
    x = x >> (!! (x >> 2) << 1);
    res = res + (!! (x >> 1) << 0);

    return res;
}

```

13、float_neg

实现：浮点数符号由最高位决定，检查是否为 NaN，否则直接翻转符号位

```

unsigned float_neg(unsigned uf) {
    /*
     * 分别获取阶码位和尾数位
     * 如果阶码位为全 1 且尾数为不为 0
     * 说明是 NaN，直接返回
     * 否则只需将最高位的符号位取反，就是结果
     */
    unsigned res = uf;
    unsigned mask_significand = ~1U >> 9;
    unsigned mask_exponent = 0xff << 23;
    unsigned exponent = mask_exponent & uf;
    unsigned significand = mask_significand & uf;
    if(exponent == mask_exponent && significand)
        res = uf;
    else
        res = uf ^ (1 << 31);

    return res;
}

```

14、float_i2f

实现：按照 IEEE 754 格式分别计算符号位、阶码和尾数

思路：首先处理符号位，若输入为负数则记录符号并取其绝对值。通过循环计算整数最高位 1 的位置，以此确定阶码值。对于尾数部分，将整数右移提取 23 位有效位，并处理舍入问题：当最高位超过 24 时，根据舍入规则判断是否需要进位。最后将符号

位、阶码和尾数组合成 IEEE 754 单精度浮点数的位级表示，若输入为 0 则直接返回 0。

```

unsigned float_i2f(int x) {
    int high_bit = 0;
    unsigned exponent = 0;
    unsigned sign = 0;
    unsigned significand = 0;
    unsigned tmpx = 0;
    unsigned res = 0;
    unsigned mask_roundF = 0;
    unsigned mask_round1 = 0;
    if(x < 0){
        x = -x;
        sign = 1 << 31;
    }
    tmpx = x;
    while(tmpx){
        high_bit++;
        tmpx = tmpx >> 1;
    }
    /*
    * 计算尾数：移位使最高位在第24位，k为最高位位置
    */
    mask_roundF = -1U >> (55 - high_bit);
    mask_round1 = 1 << (high_bit - 25);
    /*
    * 处理舍入：根据k值决定是否进位
    */
    if(high_bit >= 25){
        if((x & (mask_roundF >> 1)) > (mask_round1))
            res = 1;
        else if((x & mask_roundF) == ((mask_round1 << 1) + mask_round1))
            res = 1;
    }
    significand = ((x << (32 - high_bit)) >> 8);
    significand = significand & (-1U >> 9);
    /*
    * 计算阶码：k-1 + 127
    */
    exponent = (high_bit + 126) << 23;
    /*
    * 组合结果，特殊处理x=0
    */
    res = res + exponent + sign + significand;
    if(x == 0)
        res = 0;
    return res;
}

```

15、float_twice

实现：首先判断输入是否为特殊值（NaN 或无穷大）或零（正零/负零），若是则直接返回原数。对于非规格化浮点数（阶码为 0），将尾数左移一位实现乘以 2 的效果，同时保留原符号位。对于规格化浮点数，通过将阶码加 1 来实现乘以 2 的操作，因为规格化浮点数的表示形式为 $(1+\text{尾数}) \times 2^{\text{阶码}}$ ，阶码加 1 等价于整体乘以 2。

```

unsigned float_twice(unsigned uf) {
    /*
    * 特殊值（0 或 NaN）返回原值
    * 非规格化数左移尾数
    * 规格化数增加阶码
    */
    unsigned res = 0;
    unsigned sign = uf & (1 << 31);
    unsigned exp = (uf >> 23) & 0xff;
    if(uf == 0 || uf == (1 << 31) || exp == 0xff)
        res = uf;
    else if(exp == 0)
        res = (uf << 1) | sign;
    else
        res = uf + (1 << 23);
    return res;
}

```

三、实验过程与分析

出现问题：在编写函数完成后 测试时发现得分为 67 分，在 fitsBits 函数处发现了错误。

Correctness Results			Perf Results		
Points	Rating	Errors	Points	Ops	Puzzle
1	1	0	2	4	bitAnd
2	2	0	2	3	getByte
3	3	0	2	10	logicalShift
4	4	0	2	36	bitCount
4	4	0	2	6	bang
1	1	0	2	1	tmin
0	2	1	0	8	fitsBits
2	2	0	2	8	divpwr2
2	2	0	2	2	negate
3	3	0	2	6	isPositive
3	3	0	2	15	isLessOrEqual
4	4	0	2	45	ilog2
2	2	0	2	9	float_neg
4	4	0	2	30	float_i2f
4	4	0	2	15	float_twice

原代码为:

```
int fitsBits(int x, int n) {
    int mov1 = 16 + ~n;
    int mov2 = 17;
    return !((((x << mov1) << mov2) >> mov1) >> mov2) ^ x);
}
```

经过终端报告发现

```
ERROR: Test fitsBits(0[0x0],32[0x20]) failed
```

可以确定是在边界值 32 时发生了错误, 故修改代码 对 n=32 特殊值特殊分析, 改后成功通过。

最终得分:

Correctness Results			Perf Results		
Points	Rating	Errors	Points	Ops	Puzzle
1	1	0	2	4	bitAnd
2	2	0	2	3	getByte
3	3	0	2	10	logicalShift
4	4	0	2	36	bitCount
4	4	0	2	6	bang
1	1	0	2	1	tmin
2	2	0	2	10	fitsBits
2	2	0	2	8	divpwr2
2	2	0	2	2	negate
3	3	0	2	6	isPositive
3	3	0	2	15	isLessOrEqual
4	4	0	2	45	ilog2
2	2	0	2	9	float_neg
4	4	0	2	30	float_i2f
4	4	0	2	15	float_twice

Score = 71/71 [41/41 Corr + 30/30 Perf] (200 total operators)

故完成实验。

四、实验结果总结

实验过程中, 我首先阅读函数要求和代码规则, 根据要求逐个实现函数, 在测试过程中出现了错误, 我通过终端报告分析, 发现错误原因, 并成功改正, 最终所有函数通过了测试。

通过本次实验, 我深入理解了位操作在底层编程中的重要性, 掌握了二进制补码、浮点表示和位移技巧。并且, 让我更加清楚的认识到了, 在代码测试出错时, 可以通过观看终端报告发现错误原因, 从而更有针对性的修改代码, 同时, 我意识到要更加注重边界情况的处理, 对于边界值的处理要更加小心, 否则很容易出现错误。

五、附录（源代码）

源码截图见正文部分。